Maryland Spending Analysis Fiscal Year 2008-2026

Project Overview

This project analyzes Maryland state vendor payment data from Fiscal Year 2008 through Fiscal Year 2026. The goal of this project is to better understand how Maryland's spending is distributed across agencies, vendors, states, and contract sizes.

The main problem this project focuses on is that large government spending datasets can be difficult to understand in their raw form. Even though the data contains important information about public spending, it is not immediately clear where most payments are going, how spending has changed over time, or whether Maryland is spending more with in-state or out-of-state vendors.

To solve this, I cleaned and enriched the dataset using Python, added location data based on vendor ZIP codes, created new columns for analysis, and built a Tableau dashboard to make the results easier to explore visually.

Dataset Summary

The dataset used for this project was the **State of Maryland Payments Data** from Maryland Open Data. The dataset contains payment records from Maryland state agencies to vendors.

- Rows: **394,817**
- Columns: **8 original columns**
- Final columns after enrichment: **18**
- Time period: **Fiscal Year 2008 to Fiscal Year 2026**
- Source: Maryland Open Data

https://opendata.maryland.gov/Budget/State-of-Maryland-Payments-Data-FY2008-to-FY2024/7syw-q4cy/about_data

Key original features included:

- Fiscal Year
- Agency Name
- Vendor Name
- Vendor Zip
- Amount
- Fiscal Period
- Date
- Category

After cleaning and enrichment, additional fields were added, including:

- Vendor state
- Vendor county
- Vendor city
- In-state vs. out-of-state status
- Neighboring state status
- Contract amount bin

Data Cleaning and Preparation using Python

I began the project by loading the dataset into Python using pandas. The first step was to inspect the structure of the data using .head(), .shape, .dtypes, .nunique(), and .isna().sum().

The dataset had almost 400,000 rows and 8 columns. It included payments from **145 agencies** working with over **60,000 vendors**, which made it large enough to analyze spending patterns across many parts of Maryland government.

Column Standardization

The original column names used spaces and capital letters, so I converted them into snake case. This made the dataset easier to work with in Python.

For example:

- Fiscal Year became fiscal_year
- Agency Name became agency_name

I also stripped extra spaces from text columns to avoid issues during filtering, grouping, and merging.

```
budget_data.columns = budget_data.columns.str.replace(" ","_")
budget_data.columns = budget_data.columns.str.lower()
```

```
budget_data.columns
```

```
Index(['fiscal_year', 'agency_name', 'vendor_name', 'vendor_zip', 'amount',
       'fiscal_period', 'date', 'category'],
      dtype='object')
```

```
text_cols = ['agency_name', 'vendor_name', 'vendor_zip', 'category']
for col in text_cols:
    budget_data[col] = budget_data[col].str.strip()
```

Fixing Data Types

The amount and date columns were originally stored as object/string values. This was a problem because the amount column needed to be numeric for calculations, and the date column needed to be in datetime format for time-based analysis.

To fix this:

- I removed dollar signs and commas from the amount column.
- I converted the amount column into a numeric column.
- I converted the date column into datetime format.

After the conversion, I checked the data types again to make sure the changes worked correctly.

```
budget_data.dtypes
```

```
fiscal_year       int64
agency_name      object
vendor_name      object
vendor_zip       object
amount           object
fiscal_period     int64
date             object
category         object
dtype: object
```

```
The amount column needs to be converted to a numeric type so it can be aggregated, and the date column should be
in datetime format so time based patterns can be analyzed more easily if future records include more specific
transaction dates beyond the first of the month
```

```
budget_data['amount']=pd.to_numeric(budget_data['amount'].replace(r'[$,]','',regex=True), errors = 'coerce')

budget_data['date'] = pd.to_datetime(budget_data['date'],
    format='%Y %b %d %I:%M:%S %p',errors='coerce')
```

verifying that the data types changed

```
budget_data.dtypes
```

```
fiscal_year               int64
agency_name              object
vendor_name              object
vendor_zip               object
amount                  float64
fiscal_period             int64
date             datetime64[ns]
category                 object
dtype: object
```

Vendor ZIP Code Cleaning

One major cleaning issue was the vendor_zip column. A valid U.S. ZIP code should usually be five digits, but this column had several inconsistent values.

Some values were:

- 1 digit
- 2 digits
- 3 digits
- 4 digits
- 7 characters
- Blank values
- XXXXX
- Canadian postal code prefixes

I checked the length of each ZIP code value to find which records needed attention.

```
budget_data['vendor_zip'].str.len().value_counts()
```

```
vendor_zip
5.0    386294
4.0      8293
3.0       190
7.0        24
1.0         4
2.0         1
0.0         1
Name: count, dtype: int64
```

Handling Invalid ZIP Codes

Some ZIP codes were clearly missing or unusable. For example, values like 0, (Blank), and XXXXX were treated as missing values because they did not represent a real ZIP code.

For some vendors, I was able to fix ZIP codes using patterns already present in the data. For example, Bancroft Neuro Health had repeated records with a consistent ZIP code of 19195, so the missing 0 values for that same vendor were replaced with 19195.

For other vendors, it did not make sense to guess. Those values were left as null instead of forcing an unsupported correction.

For Bancroft Neuro Health, this vendor is listed three times outside of where the vendor zip code is 0. Because the zip code is consitent before and after the dates of the missing zip codes, it is reasonable to fill the zeros with 19195 because it is based on a pattern present in the data

```
bancroft_mask = ((budget_data['vendor_name'] =='BANCROFT NEURO HEALTH') &
(budget_data['vendor_zip'] =='0'))

budget_data.loc[bancroft_mask, 'vendor_zip'] ='19195'
```

```
budget_data[budget_data['vendor_name']=="BANCROFT NEURO HEALTH"]
```

```
budget_data['vendor_zip']= budget_data['vendor_zip'].replace('(Blank)',pd.NA)
```

```
budget_data[budget_data['vendor_zip'].str.len()==7].value_counts().sum()
```

```
0
```

All values for vendor_zip that were previously 7 character are now cleaned

Handling Short ZIP Codes

Some four-digit ZIP codes were missing a leading zero. Since ZIP codes can lose leading zeros when imported as numbers, I added the missing zero back to four-digit ZIP codes.

For some three-digit ZIP codes, I handled them case by case. The value 802 was changed to 00802 because that matched a real ZIP code pattern. Other unclear three-digit numeric values were set to null.

Canadian-style postal code prefixes, such as letter-number-letter patterns, were kept because they were not U.S. ZIP codes but still represented real location information.

```
budget_data['vendor_zip'] = budget_data['vendor_zip'].replace('802','00802')
```

```
budget_data[budget_data['vendor_zip']=='802'].value_counts().sum()
```

```
0
```

The instances of '802' are fixed

```
three_digit_mask = ((budget_data['vendor_zip'].str.len()==3)&(budget_data['vendor_zip'].str.isdigit()))

budget_data.loc[three_digit_mask,'vendor_zip']= pd.NA
```

```
budget_data[((budget_data['vendor_zip'].str.len()==3)&(budget_data['vendor_zip'].str.isdigit()))].value_counts().sum()
```

```
0
```

```
four_digit_mask= budget_data['vendor_zip'].str.len()==4
budget_data.loc[four_digit_mask, 'vendor_zip']= "0"+budget_data.loc[four_digit_mask, 'vendor_zip']
```

```
budget_data[budget_data['vendor_zip'].str.len()==4].value_counts().sum()
```

```
0
```

Data Enrichment

The original Maryland payment dataset was useful, but it was limited because it only included vendor ZIP codes, not vendor states or cities. To make the analysis stronger, I merged the payment data with a ZIP code reference dataset.

This allowed me to add:

- Primary city
- State
- County

- Timezone
- Latitude
- Longitude
- Country

Zip Code Source: https://www.unitedstateszipcodes.org/zip-code-database/

Before merging, I converted the ZIP code column in the reference dataset to a string so it would match the vendor_zip column in the Maryland spending dataset.

```
location_details['zip']= location_details['zip'].astype(str)
```

```
budget_location = pd.merge(left=budget_data, right=location_details, how='left', left_on='vendor_zip', right_on='zip')
```

```
budget_location.head()
```

	fiscal_year	agency_name	vendor_name	vendor_zip	amount	fiscal_period	date	category	zip	type	...	unacceptab
0	2008	AID TO UNIVERSITY OF MD MEDICAL SYSTEM	U M M S	21273	9701191.00	1	2008-01-01	Vendor Payment	21273	UNIQUE	...	Bank Of
1	2008	BALTIMORE CITY COMMUNITY COLLEGE	3M PHJ3884	15250	6212.00	1	2008-01-01	Vendor Payment	15250	UNIQUE	...	Bank Of N
2	2008	BALTIMORE CITY COMMUNITY COLLEGE	4 IMPRINT	53201	8312.96	1	2008-01-01	Vendor Payment	53201	PO BOX	...	
3	2008	BALTIMORE CITY COMMUNITY COLLEGE	A J STATIONERS	21226	21810.09	1	2008-01-01	Vendor Payment	21226	STANDARD	...	
4	2008	BALTIMORE CITY COMMUNITY COLLEGE	A W A MECHANICAL INC	21228	30195.57	1	2008-01-01	Vendor Payment	21228	STANDARD	...	

After merging, most records matched successfully. Some records still had missing location fields because their ZIP codes were missing, outdated, incomplete, or not found in the reference dataset. Since the unmatched rows made up a small share of the dataset, this did not seriously limit the analysis.

Feature Engineering

After cleaning and enriching the dataset, I created new columns to support analysis.

In-State vs. Out-of-State

I created an in_vs_out column to classify each payment as either:

- In State
- Out of State

This helped compare how much spending went to Maryland vendors versus vendors outside Maryland.

Neighboring State Status

I also created a neighbor_status column to classify vendors as:

- Maryland
- Neighboring State
- Non-Neighboring State

Neighboring states included Pennsylvania, Delaware, Virginia, West Virginia, and Washington, D.C. Maryland was kept as its own category so it would not be incorrectly grouped as a neighboring state.

Contract Amount Bins

I created an amount_bin column to group payment amounts into categories:

- Below 1k
- 1k to 10k
- 10k to 100k
- 100k+

This made it easier to understand the distribution of small, medium, and large payments.

Making an in state vs out of state column:

```
budget_location['in_vs_out'] = budget_location['state'].apply(lambda x: 'In State' if x == 'MD' else 'Out of State')
```

Making a Neighboring State vs Non-Neighboring State column:

```
neighbor_states = ['PA','DE','V','WV','DC']

def classify_states(state):
    if state == 'MD':
        return 'Maryland'
    elif state in neighbor_states:
        return 'Neighboring State'
    else:
        return 'Non-Neighboring State'

budget_location['neighbor_status']= budget_location['state'].apply(classify_states)
```

By definition, Maryland is technically not a neighboring state to itself, classifying Maryland as "non-neighboring" would create a misleading interpretation of the column when comparing local vs out of region spending

Making an amount bin column:

```
budget_location['amount_bin'] = pd.cut(budget_location['amount'],
                                       bins=[0, 1000, 10000, 100000, float('inf')],
                                       labels=['Below 1k', '1k to 10k', '10k to 100k','100k+'],
                                       right=False)
```

```
budget_location[['amount','state','in_vs_out','neighbor_status','amount_bin']].head()
```

	amount	state	in_vs_out	neighbor_status	amount_bin
0	9701191.00	MD	In State	Maryland	100k+
1	6212.00	PA	Out of State	Neighboring State	1k to 10k
2	8312.96	WI	Out of State	Non-Neighboring State	1k to 10k
3	21810.09	MD	In State	Maryland	10k to 100k
4	30195.57	MD	In State	Maryland	10k to 100k

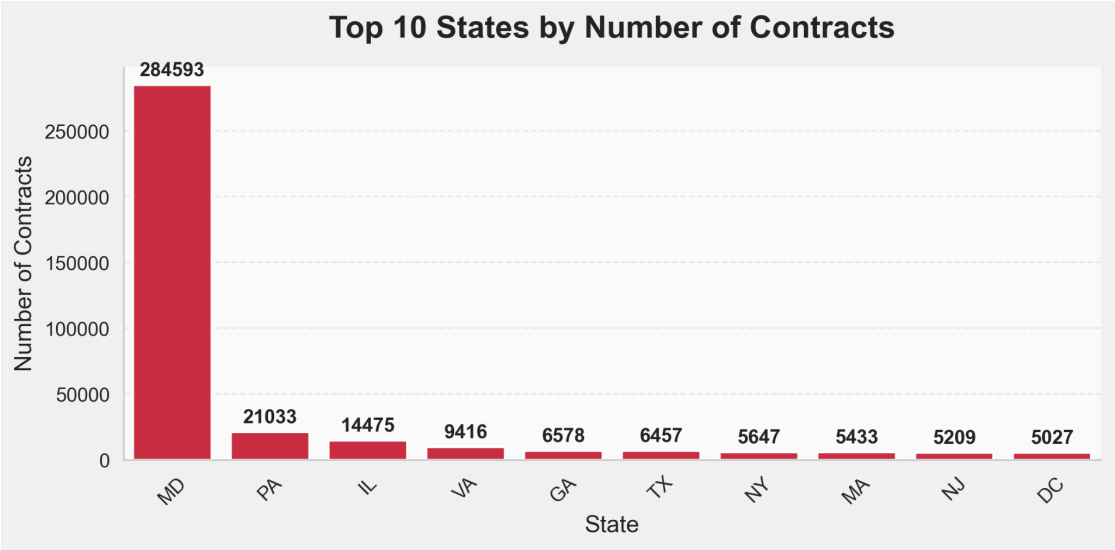
Exploratory Data Analysis

Top States by Number of Contracts

The first analysis looked at which states had the highest number of vendor payment records.

Maryland had the highest number of contracts by a large margin, with **284,593** records. The next highest states were Pennsylvania, Illinois, Virginia, Georgia, Texas, New York, Massachusetts, New Jersey, and Washington, D.C.

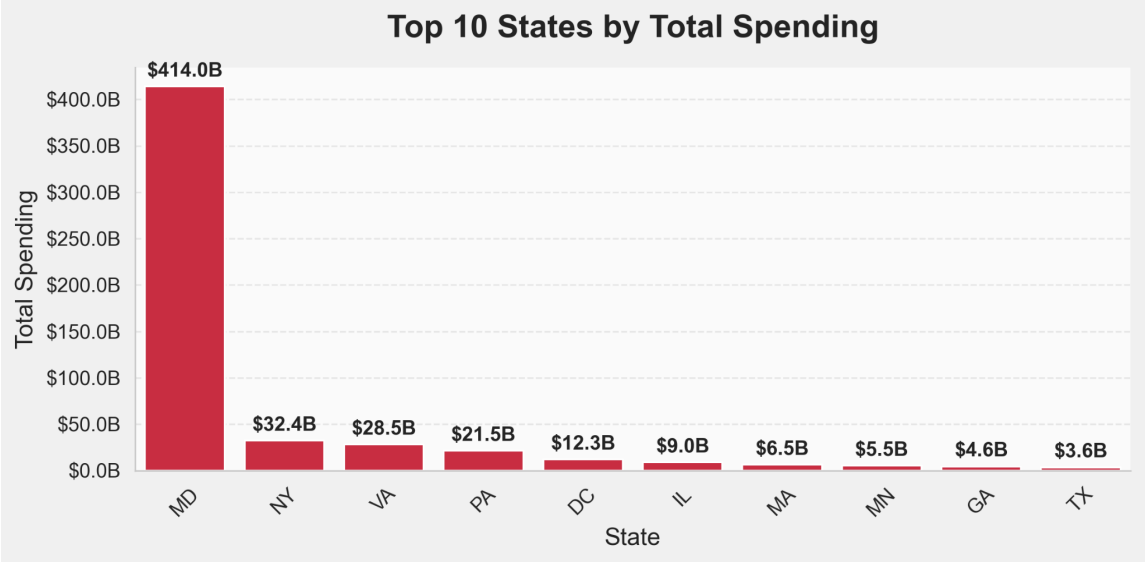
This shows that most vendor payment records are connected to Maryland vendors.



Top States by Total Spending

The second analysis looked at total spending by vendor state. Maryland again ranked first with about **$414.0 billion** in total spending. New York was second with about **$32.4 billion**, followed by Virginia, Pennsylvania, Washington, D.C., Illinois, Massachusetts, Minnesota, Georgia, and Texas.

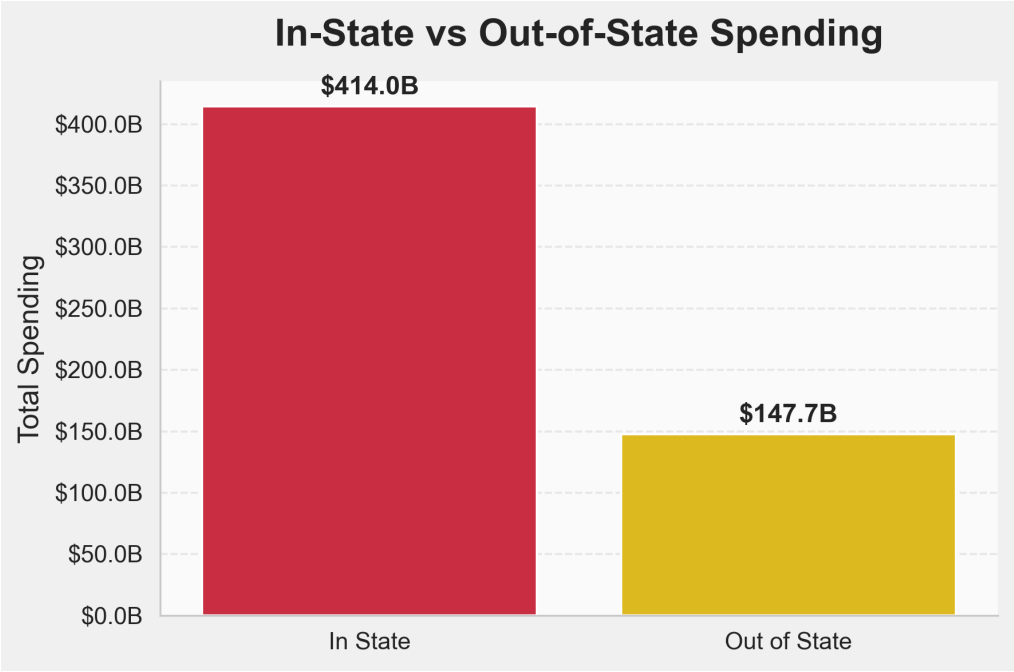
This shows that Maryland vendors not only had the most payment records, but also received the largest share of total spending.



In-State vs. Out-of-State Spending

I also compared total spending between in-state and out-of-state vendors. In-state vendors received about **$414.0 billion**, while out-of-state vendors received about **$147.7 billion**.

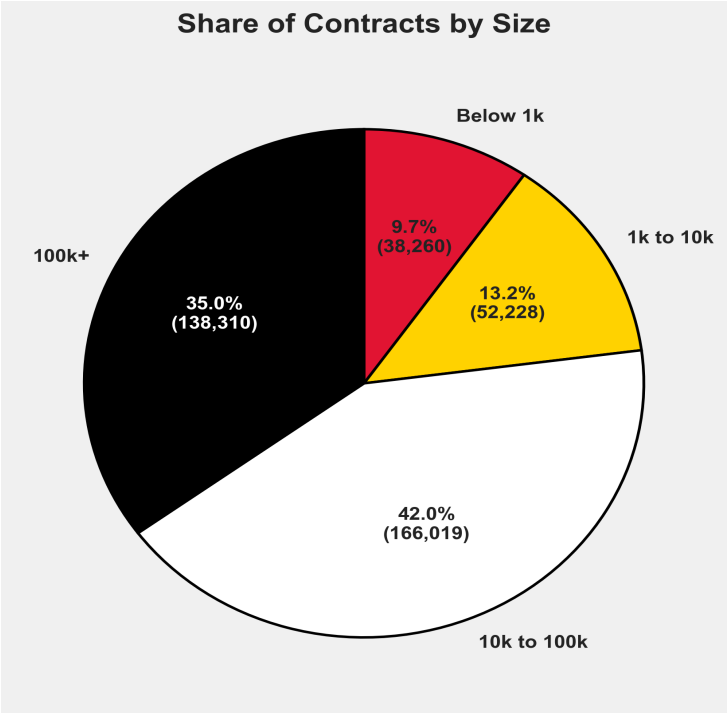
This shows that Maryland's spending is strongly focused on in-state vendors, but a significant amount of money still goes to vendors outside Maryland.



Vendor Contract Size

The contract size chart shows that most payments were not in the smallest category. The largest group was the **10k to 100k** range, which made up about **42.0%** of contracts. The **100k+** category also made up a large share at about **35.0%**.

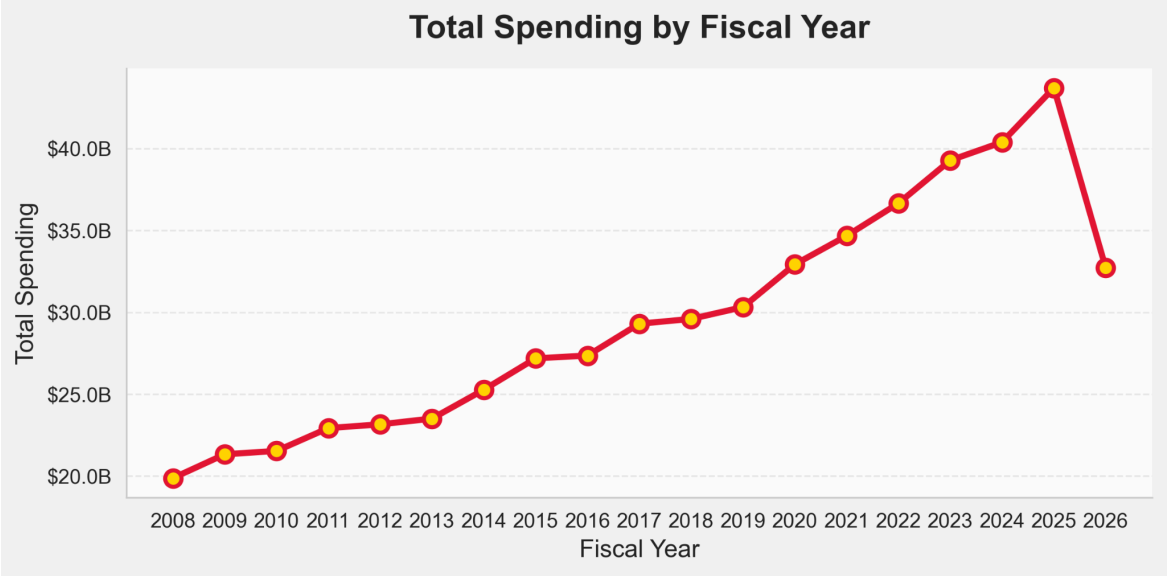
This suggests that Maryland's vendor payments are mostly made up of moderate and large payments rather than very small purchases.



Spending Over Time

The spending-over-time chart shows a steady increase in total spending from Fiscal Year 2008 through Fiscal Year 2025. Spending appears to drop in Fiscal Year 2026, but this is likely because the 2026 fiscal year data was incomplete when the dataset was downloaded.

This trend suggests that Maryland's vendor spending has generally increased over time.



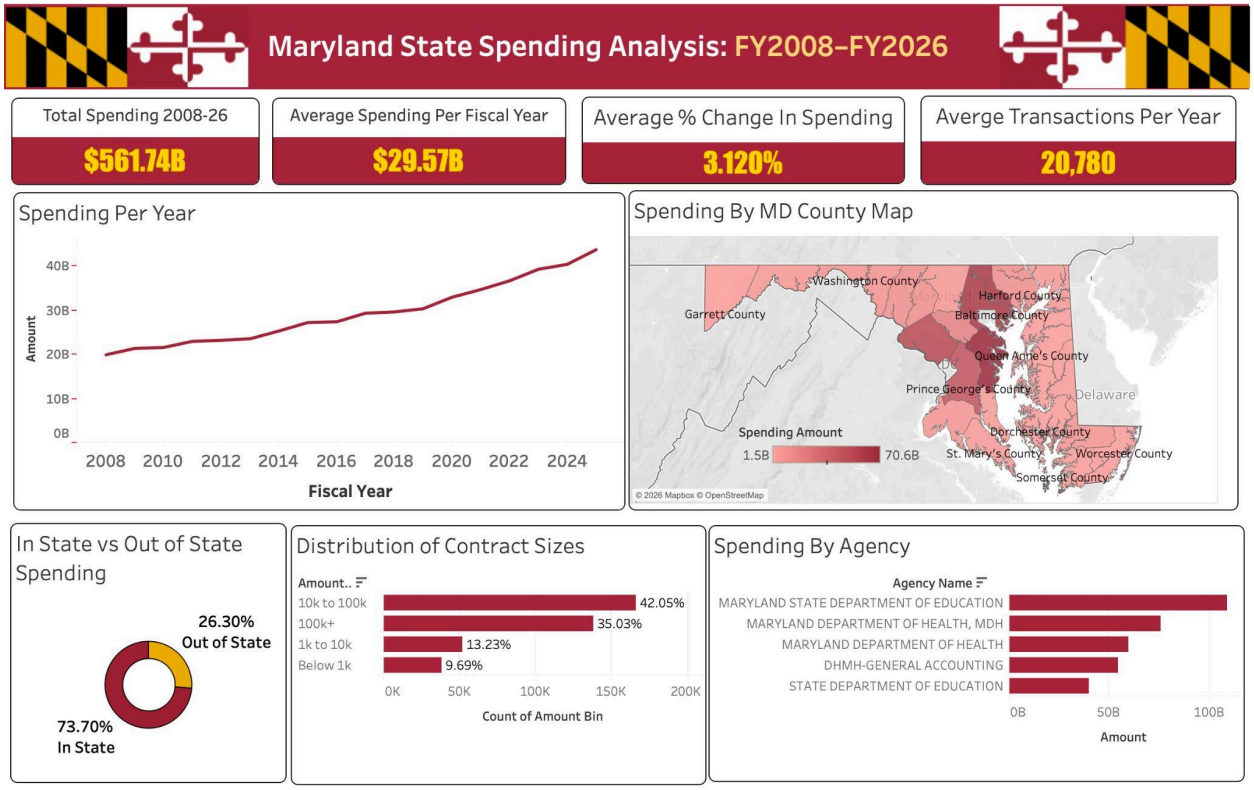
Dashboard In Tableau

Finally, I built an interactive dashboard in Tableau to present the main findings visually. The dashboard was designed to make the spending data easier to understand without needing to read through the full dataset or Python notebook.

The dashboard includes:

- Total spending from FY2008 to FY2026
- Average spending per fiscal year
- Average yearly percent change in spending
- Average transactions per year
- Spending trend over time
- Spending by Maryland county
- In-state vs. out-of-state spending
- Distribution of contract sizes
- Spending by agency

The dashboard uses Maryland-themed colors and organizes the most important metrics at the top as KPI cards. The charts below the KPIs allow users to quickly compare spending patterns across time, location, agency, and contract size.



Key Findings

Based on the analysis, several major findings stood out:

- Maryland vendors received the largest number of contracts and the largest amount of total spending.
- In-state spending was much higher than out-of-state spending.
- Vendor payments were mostly in the medium and large contract ranges.
- Total spending increased over 3% per year from 2008 through 2025.
- The 2026 drop should not be interpreted as a real decline because the fiscal year was incomplete.
- Adding ZIP code location data made the dataset more useful for geographic analysis.

Recommendations

Build a public-facing spending dashboard

Maryland should create and publish an interactive spending dashboard on its official government website to improve transparency. A dashboard like the one developed for this project would let users explore spending by fiscal year, agency, vendor, location, and payment size without needing to download and clean the raw dataset themselves. This would make the data easier for the public to understand and use.

Improve Category Code Documentation

The “category” field appears to use internal spending codes, which limits how useful the field is for public analysis. Maryland should either include a clear category reference table or plain-language category descriptions so users can better understand what each payment relates to.

Improve ZIP Code Data Entry

Data overseers should place a higher priority on ZIP code validation during data entry. Missing, invalid, or shortened ZIP codes made location matching harder, so adding stronger checks for consistent five-digit ZIP values would make it easier to connect payments to city, state, county, and map data.